

AGENTPROF: Semantic Profiling for AI Agents

Anonymous submission

Abstract

AI agents orchestrate long-running activities across users, tools, and system resources, producing growing populations of execution histories. Developers need population-level answers about where failures concentrate, which workflows trigger unsafe effects, and which task categories consume the most budget. Unlike stable code paths, natural-language intent lacks shared identity, and native execution nesting does not provide a reusable cross-run hierarchy of semantic responsibility. Existing agent tools trace runs, group metadata, and aggregate metrics, but do not combine source-linked system effects, conserved additive measures, and selectable query-time operation stacks over the same histories. Agent observability needs profiling, not only debugging. AGENTPROF realizes this insight through a *semantic operation stack model* with uniform weighted *operations*, query-time *operation stacks*, pluggable field and stack construction, label-free recurrence at coarse and detailed visible-action resolutions, and pprof-compatible output. We evaluate AGENTPROF on a 20-task real Codex capture suite, 405 human-staged CodeTraceBench trajectories, three complete public localization benchmarks, 287 OSWorld-Human sessions, and four public cost workloads. Within the declared process/tool scope, the AgentSight capture-and-join path reaches 100.0% precision and 96.6% recall and rejects 1,629/1,629 concurrent-control effects. Multi-resolution recurrence raises ordinary B^3 F1 against human stages from 0.541 to 0.663, and semantic grouping improves MAP over raw action in all three localization benchmarks. Evaluated task-family and action backends reach 0.695 and 0.498 macro-F1, label-free recurrence reaches 0.680 boundary F1 and 0.786 B^3 F1 on OSWorld-Human, and AGENTPROF constructs a 27,765-operation profile in 1.17 s.

Introduction

AI agents (Yang et al. 2024; OpenAI 2025; Anthropic 2025; Xie et al. 2024) orchestrate multi-step activities across a high-level intent layer (prompts, LLM calls, and tool invocations) and a low-level system-effects layer (process spawns and file and network I/O). As agents evolve from short scripts into complex systems, production teams accumulate many trajectories.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

Developers lack population-level answers about agent quality, safety, and cost. Which categories of tasks consume the most token budget? Where do failures concentrate across workflows? Which behavioral patterns trigger unsafe system effects? At scale, manual trajectory inspection is slow and expensive (Lù et al. 2025), while LLM judging requires a separate evaluator pass per trajectory (Lù et al. 2025; Zheng et al. 2023).

Agent trajectories lack two structures needed for cross-run profiling. Responsibility lies in semantic categories such as task intent and tool operation rather than stable function names. Natural-language prompts provide no common identifier for shared intent. Native execution nesting may exist, but it does not provide a reusable cross-run hierarchy of semantic responsibility. A prompt sequence may represent multiple tasks, while one task may belong to a larger task. Thus adapting profiling requires stable semantic identifiers and an attribution hierarchy that does not depend on a run-time stack.

Existing agent observability lacks this profiling abstraction. Tools provide rich tracing, metadata aggregation, and population dashboards (LangChain 2024; Langfuse 2024; OpenTelemetry 2024; Datadog 2025), and some derive hierarchical categories across traces and aggregate latency, cost, or evaluation metrics (LangChain 2026; Datadog 2026), while recent research canonicalizes actions into process profiles or cross-run graphs for monitoring and recovery (Shu et al. 2026; Liu et al. 2026a; Zhong, Saxena, and Raghunathan 2026; Nian et al. 2026). These systems establish semantic grouping, metric rollups, and cross-run analysis. They do not combine source-linked agent and system effects, conservation of arbitrary additive measures, and selectable query-time operation stacks over the same histories.

Agent observability needs profiling, not only debugging. Profiling applies to agents when a profiler projects resource measures onto stable semantic tags in an attribution hierarchy. Our *semantic operation stack model* provides this structure through operations and operation stacks. *Operations* are uniform records with string fields and additive measures that represent prompts, LLM calls, tool invocations, and system effects without type-specific objects. *Operation stacks* replace the runtime call stack by ordering selected fields. Changing those fields attributes the same operations by task category, execution phase, session, or action type (Figure 2).

We implement this model in AGENTPROF, an offline profiler that compiles local agent trajectories into pprof-compatible profiles (Google 2024) (Figure 1). *Intent attribution* derives stable, low-cardinality tags from natural-language fields via regex rules, a local LLM tagger, or unsupervised clustering, replacing function-name identity. *Stack construction* uses either selected fields or label-free recurrence at coarse and detailed visible-action resolutions to replace runtime nesting.

Within the declared process/tool scope, AgentSight (Zheng et al. 2025) capture and joining on 20 real Codex tasks reach 100.0% precision and 96.6% recall and reject all 1,629 concurrent-control effects. Across all 405 reconstructable failed CodeTraceBench trajectories (Li et al. 2026), multi-resolution recurrence raises ordinary B³ F1 against human stages from 0.541 to 0.663 over raw action. Across three complete public workloads, semantic grouping raises MAP over raw action from 0.773 to 0.789, 0.281 to 0.452, and 0.121 to 0.230, respectively (Fan et al. 2026; Wang et al. 2026; Chen et al. 2026a). On 287 OSWorld-Human development sessions (Abhyankar, Qi, and Zhang 2025), label-free recurrence reaches 0.680 boundary F1 and 0.786 B³ F1. On the AgentBoard and ASE populations, task-family and action backends reach 0.695 and 0.498 macro-F1, respectively (Ma et al. 2024; Bouzenia and Pradel 2025). On four public workloads, AGENTPROF constructs a 27,765-operation profile in 1.17 s.

This paper makes three contributions:

1. **Semantic operation stack model.** Uniform operations and query-time stacks supply agent observability’s missing profiling layer (Background and Motivation; Design).
2. **System: AGENTPROF.** Pluggable intent attribution and stack construction produce pprof-compatible profiles (Implementation).
3. **Evaluation.** Semantic profiles preserve linked effects and improve attribution and problem ranking over raw action. Field evaluations cover task partitions, the integrated task-family path, a standalone action-label adapter, and session-held-out groups. AGENTPROF profiles 27,765 operations in 1.17 seconds (Evaluation).

Background and Motivation

LLMs and AI Agents

A typical agent trajectory interleaves two layers of activity (Yang et al. 2024; OpenAI 2025; Anthropic 2025; Zheng et al. 2025). At the intent layer, an agent receives prompts, issues LLM calls, and invokes tools for code execution, search, and file editing. Tool invocations trigger lower-layer process, file, and network effects (Zheng et al. 2025). Trajectories repeat this cycle, and teams accumulate many trajectories.

System Profiling

Unlike single-execution debugging, profiling aggregates measurements to attribute resources to responsible entities. Traditional profilers sample execution, attach each sample to its call stack, and fold identical stacks (Google 2024; Gregg 2011). Flame graphs (Gregg 2011) and pprof (Google

2024) call graphs visualize aggregate cost by width or node size, while Pivot Tracing (Mace, Roelke, and Fonseca 2015) dynamically selects and groups measurements across causally related events. These tools start from stable code identifiers and runtime stacks, although pprof can also promote `tagroot/tagleaf` values to pseudo-frames (Google 2024). Agent profiles must instead derive stable semantic fields and connect them to downstream effects.

Challenges for Agent Profiling

Existing tools (LangChain 2024; Langfuse 2024; OpenTelemetry 2024) record agent events as per-execution span trees for single-run debugging. AgentSight (Zheng et al. 2025) bridges intent and system-effect layers by correlating extended Berkeley Packet Filter (eBPF) system events with intercepted LLM traffic, but the resulting recordings still lack aggregate profiling.

Agent profiling faces two challenges. First, responsibility lies in semantic categories such as task intent and workflow phase, not code paths. Standards represent application-supplied span attributes (OpenTelemetry 2024; Arize AI 2024), but natural-language histories lack stable categories because different prompts or tools can express the same intent.

Second, agent trajectories lack a runtime call hierarchy for semantic attribution. In CPU profiling, the stack attributes `malloc`’s cost simultaneously to `parse`, `process`, and `main` (Google 2024; Gregg 2011). A prompt sequence may contain one or several tasks within a larger workflow, yet no runtime mechanism marks its boundaries or attribution granularity. RQ1 evaluates how this missing hierarchy affects attribution, while RQ3 evaluates constructed groups and boundaries.

Design Requirements for Agent Profiling

Traditional profiling relies on three mechanisms that the call stack supplies automatically. Agent profiling requires all three without a call stack:

- D1: Cross-layer resource projection.** The profiler must connect resource consumption from file I/O and process spawns to the responsible prompts and LLM calls.
- D2: Stable tags for folding.** Without stable function names, the profiler must derive short, repeatable tags from natural language before folding.
- D3: Hierarchical attribution.** Because span trees do not encode semantic responsibility through function-call nesting, the profiler must construct attribution hierarchies from the data.

Design

The design maps D1–D3 to ingestion, tags, and operation stacks. For D1, source-specific capture and adapters preserve each system effect’s link to its responsible intent, and operations carry the inherited semantic fields into projection and folding. For D2, intent attribution derives stable tags from natural-language fields. For D3, *operation stacks* replace runtime stacks with query-time field projections, supporting multiple attribution granularities over the same data.

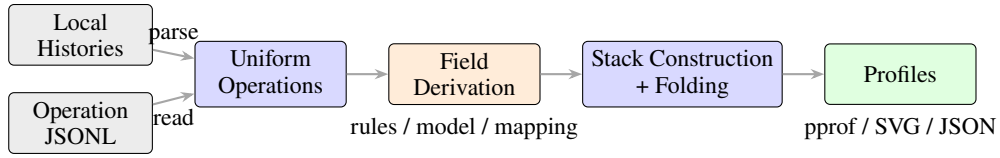


Figure 1: AGENTPROF pipeline. Local histories enter through native history parsing, whereas AgentSight adapters and public datasets enter as newline-delimited JSON operation records. All yield uniform operations before field derivation, stack construction, folding, and profile export.

The profiling pipeline has four stages: (1) parse raw trajectories into operations, (2) enrich operations via intent attribution or mapping rules, (3) project each operation onto a stack frame sequence chosen at query time, and (4) fold operations with identical stacks, summing their weights.

Semantic Operation Stack Model

AGENTPROF represents each activity from the intent and system-effect layers as an *operation*, a weighted record with string fields and additive measures. Each operation carries string fields (`project`, `agent`, `session`, `prompt_tag`, `kind`, `model`, `path`, `domain`, `status`, ...) and additive measures (token count, duration, event count). The same schema represents prompts, LLM calls, tool invocations, file reads, GUI clicks, and process events. Populated fields distinguish these operation types. When a tool invocation triggers system effects, ingestion propagates its semantic fields to the resulting operations, so AGENTPROF can fold downstream resource weights under the responsible intent.

An *operation stack* replaces the runtime call stack to provide hierarchical attribution for agent trajectories. In CPU profiling, a call stack determines which code paths share responsibility for a resource cost. In AGENTPROF, an ordered list of fields $[f_1, f_2, \dots, f_k]$ projects each operation o to a frame sequence $\langle o.f_1, o.f_2, \dots, o.f_k \rangle$, and operations with identical sequences are merged, summing their weights. Unlike a call stack, the fields are chosen at query time, not determined by execution. The same operations can therefore support a flat task summary, a per-session tree, a semantic phase/action hierarchy, or a trajectory organized by the dataset’s own annotations. Field selection changes attribution but not the data. Sessions and spans are optional fields, not fixed hierarchy levels. The same data supports debugging views with `session` and aggregate profiling views without that field.

We formalize each selectable profile as a view (φ, σ, w) . For operation o , the predicate $\varphi(o)$ decides inclusion, an ordered field list $[f_1, \dots, f_k]$ defines $\sigma(o) = \langle o.f_1, \dots, o.f_k \rangle$, and $w(o) \geq 0$ supplies its weight. Users choose the stack function independently of the predicate and weight function. For common resource questions, four built-in choices instantiate φ and w : (1) `tokens` weights LLM calls by token count, (2) `time` weights timed operations by duration, (3) `files` weights file operations by event count, and (4) `network` weights network operations by event count.

Attribution and Stack Construction

Intent attribution. Intent attribution maps natural-language fields to short, stable, repeatable tags, supplying the identifiers that traditional profiling obtains from function names. The framework supports three pluggable backends. Explicit rules provide repeatable user control, a local model tags natural-language fields, and unsupervised grouping discovers candidate categories that developers can encode as rules. Structured trajectories use the same interface to derive fields from existing action and quality attributes. All backends emit ordinary operation fields, keeping the stack model independent of how a tag was obtained.

Stack construction. With an ordered field list, stack construction directly projects the available fields. When users enable automatic construction, label-free recurrence lets AGENTPROF induce operation stacks without a user-specified field list. Visible actions are source-field values available before target annotations are read. The constructor learns adjacent recurrence across reference sessions at a coarse action resolution and, when complete detail is available, at the compound (*action*, *action_detail*) resolution. A transition continues the segment when either resolution supports continuity. Weak or unseen transitions at both resolutions start a segment. Each segment’s run-length-compressed coarse-action sequence becomes a reusable stack-frame value. The derived group and stack-frame values are ordinary operation fields, so the stack function can compose them with task, phase, action, or session fields at query time. The constructor can learn from a separate label-free operation corpus, so it does not use the target session’s problem annotations.

Implementation

AGENTPROF is an offline Rust command-line interface that implements the semantic operation stack model (Figure 1).

Input reconstruction. AGENTPROF reads local Codex and Claude Code histories in JSON Lines (JSONL) format and also accepts operation JSONL. For native histories, it reconstructs prompts, LLM calls, and tool invocations. Because the current CLI does not read AgentSight (Zheng et al. 2025) recordings directly, a source-specific adapter converts those recordings to operation JSONL. For AgentSight recordings, the declared process/tool scope contains the captured target agent process and its launched tool command, excluding concurrent control activity. AgentSight joins system effects within this scope, and the adapter emits operations carrying those links. AGENTPROF then projects and folds their fields and weights.

Field derivation. The production field-derivation path uses first-match regex rules of the form `KIND : TAG=REGEX`, which developers refine by rerunning `AGENTPROF` and inspecting unmatched operations. A local LLM tagger runs a quantized 3B-parameter model through `llama.cpp` (Gerganov and contributors 2026) to generate a single-word tag or select from a user-declared task-tag set. Both modes use grammar-constrained decoding and caching. TF-IDF (Salton and Buckley 1988) and K-Means (MacQueen 1967) discover candidate prompt groups without predefined rules, while mapping rules derive fields for structured trajectories, for example `phase:navigate=type:(click|scroll|key)`.

Boundary construction. The Rust inducer counts adjacent action transitions in reference sessions and scores each transition by normalized pointwise mutual information (NPMI), $NPMI(a, b) = \ln[p(a, b)/(p_L(a)p_R(b))]/-\ln p(a, b)$, with all three probabilities defined over the same transition sample space (Bouma 2009).

Occurrence-weighted one-dimensional k -means with $k = 2$ (MacQueen 1967) initializes at the minimum and maximum scores, assigns distance ties to the lower center, and uses the converged midpoint as a global cutoff. The inducer applies the same procedure to action-changing transitions for a second cutoff. Same-action pairs use the global cutoff, whereas different-action pairs use the smaller cutoff, which can remove a boundary produced by the global rule but cannot add one. When every reference and target operation also has a nonempty `action_detail`, the inducer fits the same recurrence model to the compound signature. Detailed continuity may remove a coarse boundary but can never add one. Missing, unseen, or weak detail falls back exactly to the coarse decision. Segment names remain coarse run-length-compressed action sequences. The inducer reads only session identity, input order, action, and optional action detail, not target annotations or resource weights.

An optional reference-calibrated recurrence mode uses the same NPMI score but selects one scalar cutoff that maximizes per-operation B^3 F1 on disjoint reference groups. The mode enumerates score intervals, resolves exact ties toward the smallest cutoff, and never reads target groups. Label-free k -means remains the default.

Profile export. `AGENTPROF` emits one pprof protobuf profile for existing tools such as `go tool pprof`; it does not add a visualization frontend or a second export format.

Evaluation

Research questions. The evaluation addresses four research questions: **RQ1:** Does semantic profiling improve resource attribution? **RQ2:** Does profiler output correspond to real problems? **RQ3:** How accurate are the tags? **RQ4:** What is the profiling cost? Each RQ subsection pairs its protocol with the strongest answer its evidence supports.

We evaluate three data classes separately. **Real inputs (RQ1)** include 325 local histories (198 Codex, 50 Claude, and 77 Claude subagent) with 183,714 system observations collected over multiple months and a separate 20-task controlled capture suite. **Annotated trajectories (RQ1, RQ3,**

RQ4) include 15 real-agent or human web, API, coding, and mobile/GUI execution families mapped into 47,590 operations, plus 405 CodeTraceBench trajectories with 20,866 operations (Lù, Kasner, and Reddy 2024; Deng et al. 2023; Xu et al. 2025; Yao et al. 2022; Li et al. 2023; Qin et al. 2024; Yao et al. 2025; Yang et al. 2024; Li et al. 2024; Lu et al. 2025; Liu et al. 2026b; Lù et al. 2025; Chen et al. 2026b; Wang et al. 2025; Abhyankar, Qi, and Zhang 2025). **Problem-localization benchmarks (RQ2)** provide independent targets over 27,346 operations (Fan et al. 2026; Wang et al. 2026; Chen et al. 2026a). The RQ4 union contains 27,765 operations (Lù et al. 2025; Chen et al. 2026b; Abhyankar, Qi, and Zhang 2025; Wang et al. 2025).

RQ1: Resource Attribution

RQ1 asks whether semantic profiling improves resource attribution beyond what traditional tag-free tools provide.

We first test whether the capture-and-join path assigns system effects within the declared process/tool scope while rejecting concurrent controls. We use `AgentSight 0.2.37` to capture 20 real Codex task executions, each paired with one concurrent control, and an adapter to convert joined effects into operation JSONL for `agentpprof 0.2.37`.

The controlled suite establishes scoped capture-and-join accuracy and lossless folding with predeclared rather than inferred task categories. All task and control runs completed. The path recovers 1,520/1,574 in-scope effects with no false positives (100.0% precision, 96.569% recall), rejects all 1,629 control effects, and emits 1,520 samples in 152 stacks while preserving the input and all five predeclared task-category totals.

We next test whether the current operation stack partitions operations into independently annotated responsibility units more faithfully than raw-action grouping. We use all 405 failed CodeTraceBench trajectories whose released sources yield the official operation sequence. The population contains 20,866 operations and 2,948 human-verified contiguous stage intervals across OpenHands, mini-SWE-agent, SWE-agent, and Terminus2 (Li et al. 2026). The recurrence constructor reads neither target stages nor resource weights. It learns from target-disjoint reference sessions and uses visible action kind and raw-action detail as its coarse and detailed resolutions, respectively. Matched control methods use raw-action changes as boundaries, group by the named phase or action-kind source field, place each session in one group, or make every operation a singleton. Every method receives the same operations and stages.

Every method partitions the same weighted operations, and folding conserves their additive weights. We therefore use ordinary operation-level B^3 precision, recall, and F1 (Bagga and Baldwin 1998) to test whether weights reach the intended responsibility units.

Multi-resolution recurrence improves agreement with human responsibility partitions over raw-action grouping, raising ordinary B^3 F1 from 0.541 to 0.663 (Table 1). Relative to the coarse constructor, its gain is 0.014 with a task-clustered paired-bootstrap 95% interval of [0.009, 0.018], and the effect is positive in all four agent frameworks.



Figure 2: Semantic flame graphs of 325 real agent trajectories. Width represents token count (top), summed per-operation elapsed duration (middle), and the number of file operations (bottom). All three aggregate the same operation corpus and differ only in included operations, stack fields, and the additive width measure.

Method	B ³ P	B ³ R	B ³ F1
Multi-resolution recurrence	0.782	0.575	0.663
Coarse recurrence	0.829	0.534	0.649
Raw-action grouping	0.891	0.388	0.541
Phase only	0.686	0.626	0.654
Action kind	0.948	0.315	0.473
Per session	0.174	1.000	0.296
Per operation	1.000	0.141	0.248

Table 1: RQ1 agreement with human responsibility partitions on 405 CodeTraceBench trajectories. Ordinary B³ is the standard partition-agreement metric.

Phase-only reaches 0.654 ordinary B³ F1, compared with 0.663 for multi-resolution recurrence. The multi-resolution result supports recurrent semantic responsibility partitioning over raw-action grouping on this population. Because CodeTraceBench influenced constructor selection, this is post-hoc mechanism evidence rather than independent confirmation or a claim that one profile dominates every decision. RQ3 separately tests the detail-free fallback on OSWorld-Human.

As a separate use case, five declared field hierarchies project the same 13,265 operations from nine datasets into 9 dataset, 57 phase, 226 semantic, 455 action, or 3,757 per-session groups. They expose task budgets, recurring actions, or session detail without changing the operation corpus. Adapters map all 15 annotated families into this common model without type-specific objects.

Changing only the selection predicate, stack fields, and weight function produces the `tokens`, `time`, and `files` profiles in Figure 2. Separately, prompt-span duration and system-effect-count rankings share 7 of their top 10 prompt categories (Spearman $\rho = 0.623$), with `network`-tagged

prompts ranked 8th and 93rd, respectively. Prompt-span duration measures elapsed rather than active CPU time and may include idle or user wait. Together, RQ1 shows that scoped capture and joining identify effects with the reported precision and recall, lossless folding preserves joined effects, semantic responsibility partitions improve ordinary B³ over raw action, and selectable stacks and weights expose different views over the same operations. RQ3 separately evaluates field, group, and boundary construction.

RQ2: Problem Correspondence

RQ2 asks whether target-blind profiles place independently annotated problems in groups a developer can inspect early. We run complete AgentProcessBench, HINTBench, and TraceElephant workloads (Fan et al. 2026; Wang et al. 2026; Chen et al. 2026a). Of HINTBench’s reported 629 trajectories, the released test snapshot used here contains 536. A separate 80-trajectory validation snapshot selects one of 24 field orders before all 536 tests are scored.

Within each workload, we give AGENTPROF and raw-action grouping identical operations and precomputed benchmark judge/localizer predictions, so they differ only in grouping. We fix predictions, group scores, profiles, and scoring rules before loading the corresponding test target labels. We select HINTBench’s field order on the separate validation snapshot described above. AgentProcessBench averages judge votes within each group. For HINTBench and TraceElephant, we assign every root-to-frame stack prefix the 95% Wilson lower bound (Wilson 1927) of the fraction of its members with a positive frozen benchmark prediction. Each operation inherits the maximum score among prefixes containing it, with ties preserved.

Each query is one trajectory containing an annotated target. We report non-interpolated per-query AP, evaluating ties

Workload	Sem.	Raw	Local+S	Local+R	Local
AgentProcessBench	.789	.773	.896	.893	.863
HINTBench	.452	.281	.545	.506	.411
TraceElephant	.230	.121	.322	.249	.209

Table 2: Standard per-query AP averaged as MAP after scoring every operation. Semantic versus raw action is the primary comparison. Local+semantic, local+raw, and local-only are a post-hoc mechanism analysis on the same complete populations. Higher is better.

at their shared score threshold, and arithmetic-mean MAP across the 614, 400, and 220 target-bearing queries, respectively (Robertson 2008).

The post-hoc local-only column ranks operations by the benchmark-provided diagnostic score. Local+semantic and local+raw preserve every strict local-score ordering and use the semantic or raw-action group score only to refine exact ties.

In the primary comparison, AGENTPROF improves MAP over raw action on all three complete workloads (Table 2). The gains are +.016 [.005,.027], +.171 [.154,.188], and +.109 [.077,.142], using 10,000 paired resamples of trajectory clusters within benchmark-defined strata.

The adaptive local-first results support semantic grouping as a tie-breaking refinement of, rather than a replacement for, operation-local diagnostic scores, but do not constitute untouched confirmation. In this post-hoc mechanism analysis, local+semantic MAP is .896/.545/.322, above local-only and semantic-only on all three workloads. Relative to the identically composed local-plus-raw tie-breaker, it gains +.039 [.029,.049] on HINTBench and +.073 [.050,.097] on TraceElephant. AgentProcessBench does not distinguish the two refinements (+.003 [-.0005,.0069]).

Case Study 1: Auditing Repeated Reviews

We next ask whether a semantic profile can answer concrete user questions that are not visible in a flat tool/action log. After confirming only that the profile was readable and conserved its input, we fixed four questions before running focused pprof queries: (1) where each review spent its effort, (2) how work changed after an initial finding, (3) whether expensive paths culminated in a review conclusion, and (4) which operations recurred across reviews versus exposed task-specific exceptions.

This collection contains four complete real AgentCap Codex sessions reviewing R024 evaluator-backed replay, R025 checker ablation, R035 gateway mismatch analysis, and R081 top-conference evidence. One Agent read all indexed operation summaries and emitted 64 sparse start-ID marks plus a shared pool of 29 operation names; regexes did not select semantic boundaries. AGENTPROF inherited each mark until the next mark in the same session and folded all 326 operations into 62 stacks of depth three to five. The resulting 7,410-byte pprof is the only profiler output; all queries below use stock `go tool pprof`.

The profile attributes 125/326 operations (38.3%) to veri-

fying requested fixes. That recurring operation decomposes into re-establishing review scope (30 operations), validating fixed artifacts (21), inspecting the proposed fix (15), auditing fixed documentation (14), running focused tests (14), and smaller task-specific branches. Across all four reviews, experiment-evidence validation accounts for 55 operations, including 22 spent recomputing reported results, nine running focused tests, seven diagnosing a test-environment failure, and four verifying result provenance.

Shared names preserve comparison without erasing exceptions. R024 contains a ten-operation evaluator-deviation branch split into action-matching semantics (seven) and a banking fallback (three). R035 contains nine operations that locate stale documentation, three of them under the blocking-finding path. R081 separately spends five operations inspecting and five validating its reference corpus. Every session eventually reports a conclusion: only eight operations belong to the terminal report nodes, while six earlier operations explicitly identify blocking findings. Thus this case does not exhibit an entire high-cost session with no conclusion; instead, the profile separates the large evidence-gathering paths from the small conclusion-bearing paths and shows which work recurs during repair.

This is a product case, not an independent nested-accuracy result: the Agent assigned the boundaries and shared names that the case queries. RQ3 evaluates independently annotated fields, partitions, and boundaries.

Case Study 2: Differential Profiling at Scale

We next aggregate the complete mixed-outcome population from AgentRewardBench (Lù et al. 2025), rather than presenting one favorable trace pair. The collection contains 440 real trajectories over 125 tasks with both successful and unsuccessful runs, producing 338 complete bad-good pairs: 24 pair occurrences from AssistantBench, 102 from VisualWebArena, 144 from WebArena, and 68 from WorkArena. Because tasks can contain multiple runs, the 202 successful and 238 unsuccessful sessions are reused across pairs; the profile is therefore pair-occurrence weighted. All 338 bad members and all 338 good members are folded into one signed operation-count pprof. Individual pair profiles are used only to descend from a collection-level path to source evidence.

The aggregate contains 7,366 bad-side and 3,780 good-side operation occurrences. Positive values denote bad-only or bad-side excess paths; negative values denote good-only or good-side excess paths. The largest bad-side result paths are continuing progress (+1,825) and exact repeated action-state work (+1,261). The latter decomposes into click (+639), no-op (+356), and scroll (+277) paths. Concrete bad-side error families remain separately searchable: clicking an invisible element (+148), click timeout (+129), stopped execution (+92), filling a non-input element (+57), and selecting on a non-select element (+46). The good side contains more terminal (-92) and conclusion (-67) paths; `send_msg_to_user` has a cumulative difference of -100.

This is diagnostic structure, not a new outcome classifier. On the same population, simple step count remains the strongest tested scalar (pairwise accuracy .7633; trajectory ROC AUC .7517). The profile adds a different answer: it

identifies whether excess work is repetition, waiting, wrong-widget interaction, timeout, stopping, or missing completion, while retaining action and object frames for stock pprof drill-down. AgentRewardBench has outcome and looping annotations but no gold semantic hierarchy, so this case establishes complete differential-profile coverage and concrete path exposure, not semantic-stack accuracy or causality.

RQ3: Tag Accuracy

RQ3 evaluates fields produced by predeclared derivation methods against independent annotations under each protocol’s stated input boundary. Literal task and action fields use accuracy and macro-F1 (the unweighted mean of per-class F1) (Lewis et al. 2004). Partition-valued task, phase, and group fields use V-measure (Rosenberg and Hirschberg 2007) or ordinary B^3 , and adjacent group boundaries use exact precision, recall, and F1 (Ruokolainen et al. 2016). Literal labels, permutation-invariant partitions, and adjacent boundaries are distinct outputs. A *tag* is an operation field consumed by AGENTPROF’s pluggable field interface. Whether run through the CLI or a standalone adapter, a backend maps its output to this field, from which AGENTPROF derives structural groups and boundaries. AGENTPROF folds predicted OSWorld groups with their original additive weights.

For the structural-field component of RQ3, we reuse the CodeTraceBench partition result reported under RQ1 in Table 1. Across 20,866 operations and 2,948 human stages, multi-resolution recurrence reaches 0.663 ordinary B^3 F1 and deterministic phase-only grouping reaches 0.654. Below we test literal labels, other partitions, and boundaries.

We test all 287 OSWorld-Human task-instance sessions (Abhyankar, Qi, and Zhang 2025) with complete group annotations. This population contains 3,978 operations, 3,691 adjacent pairs, and 2,042 groups. For the supervised Bernoulli Naïve Bayes predictor (McCallum and Nigam 1998), we fix the model family, nine input fields, adjacent-pair features, and training procedure. Each session-held-out cross-validation fold fits its parameters and threshold on training sessions and predicts every held-out session once. Predicted boundaries become an ordinary group field that AGENTPROF projects and folds. With the same five folds, label-free recurrence uses only the other sessions’ visible action transitions. OSWorld-Human supplies no non-redundant action detail, so the multi-resolution constructor exactly falls back to its coarse arm. Because we designed the recurrence rule after inspecting earlier results on this corpus, these results are development evidence rather than independent cross-family confirmation. Optional reference-calibrated recurrence uses training operations and group annotations to fit one NPMI cutoff, while held-out folds contribute only visible actions until predictions are fixed.

Controls place a boundary after every operation, visible action change, or visible phase change. We also report ordinary per-operation B^3 partition F1 (Bagga and Baldwin 1998), whose predicted–human group overlap captures merging and fragmentation.

The supervised predictor reaches 0.739 boundary F1 and 0.816 B^3 F1 versus 0.645 and 0.678 for the strongest controls. Label-free recurrence reaches 0.680 and 0.786, while

Method	Prec.	Recall	Bound. F1	B^3 F1
Supervised predictor	0.700	0.782	0.739	0.816
Reference-calibrated recurrence	0.610	0.922	0.734	0.801
Label-free recurrence	0.592	0.799	0.680	0.786
Always boundary	0.476	1.000	0.645	0.678
Action change	0.386	0.626	0.477	0.659
Phase change	0.441	0.268	0.334	0.665

Table 3: RQ3 boundary and partition agreement on OSWorld-Human. Boundary F1 scores exact adjacent-pair decisions. B^3 scores predicted–human partitions.

reference-calibrated recurrence reaches 0.734 and 0.801 but requires group annotations that the default label-free method does not use. Every method conserves the total projected weight.

For target-blind TF-IDF/K-Means, V-measure (Rosenberg and Hirschberg 2007) over operation assignments reaches 0.557 on nine Mind2Web sessions (49 operations) and 0.815 on 100 ScienceWorld sessions (2,504 operations) (Deng et al. 2023; Wang et al. 2022). Every operation receives a prediction in both datasets, whereas a constant tag yields V-measure 0. Here, V-measure tests partitions, not literal names.

For task-family tags, we use a fixed evaluation-only Qwen3.6-27B llama.cpp backend distinct from the 3B CLI backend. Using only goals and declared family descriptions, it classifies all 1,012 AgentBoard goals into nine families (Qwen Team 2026; Ma et al. 2024). The backend reaches 0.695 macro-F1 and 0.733 accuracy, compared with 0.044 macro-F1 and 0.248 accuracy for the majority-class baseline, and produces identical assignments across three runs.

For action tags, a standalone backend-level adapter runs the fixed Qwen3.6-27B closed-label configuration using only the current thought/action and eight action definitions operationalized by TraceView (Qwen Team 2026; Sajadi et al. 2026). The adapter classifies all 2,737 published annotations from 120 AutoCodeRover, OpenHands, and RepairAgent trajectories (Bouzenia and Pradel 2025). The model reaches 0.498 macro-F1 and 0.628 accuracy, compared with 0.061 macro-F1 and 0.323 accuracy for the majority-class baseline. Its 0.437 macro-F1 gain has a whole-trajectory 95% bootstrap interval of [0.380, 0.494], and both complete runs agree exactly. Thirty-nine AutoCodeRover inputs contain the target word `Locate`. Excluding them leaves 0.490 macro-F1 and 0.622 accuracy, so the positive comparison to majority is unchanged.

Thus, on the named public populations, RQ3 establishes target-blind task partitions at V-measure 0.557 and 0.815, phase structure at 0.654 ordinary B^3 F1, task-family assignment at 0.695 macro-F1, action assignment at 0.498 macro-F1, and label-free group boundaries at 0.680 boundary F1 and 0.786 B^3 F1.

RQ4: Profiling Cost

RQ4 measures the time to parse fixed-field operation JSONL, construct stacks, fold weights, and serialize the profile. The measurement excludes capture, source adaptation, and field/tag generation. Across four complete pub-

lic workloads and their union, we compare the fixed semantic hierarchy project, agent, task, phase, op, tool, status with the raw hierarchy project, agent, action, status on identical inputs over three runs. Measurements use agentpprof 0.2.37 on a 24-core Intel Core Ultra 9 285K with 125 GiB RAM and Linux 6.15.11.

Workload	Ops	Semantic (s)	Raw action (s)	Peak RSS (MiB)
AgentRewardBench	729	0.04	0.03	19.0
SATraj-OS	4,285	0.17	0.14	73.5
OSWorld-Human	6,010	0.25	0.21	108.4
AgentNet	16,741	0.71	0.58	278.4
Union	27,765	1.17	0.99	464.5

Table 4: RQ4 profile-construction cost. Times are three-run medians, and resident set size (RSS) is the largest peak observed for the semantic profile.

Both time curves are monotonic across the five input sizes. The semantic curve has a descriptive slope of 0.0422 ms per operation ($R^2 = 0.9997$), reaching 23,731 operations/s on the union. For the 27,765-operation union, semantic construction completes in 1.17 seconds, reaches 464.5 MiB peak RSS, and incurs 180 ms (18.2%) more time and 6.0 MiB (1.3%) more peak RSS than raw-action grouping.

Scope and Limitations

Results apply to the named populations, tag sets, and declared process/tool scope. CodeTraceBench influenced constructor selection, so its 405-trajectory result (Li et al. 2026) is post-hoc support. RQ2’s local-first analysis is likewise adaptive and does not replace the primary semantic-versus-raw comparison. RQ4 measures fixed-input parsing, stack construction, folding, and serialization, excluding capture, source adaptation, field/tag generation, and live-agent overhead.

Related Work

Agent observability. Platforms aggregate spans, metadata, and evaluations (LangChain 2024; Langfuse 2024; OpenTelemetry 2024; Datadog 2025). Datadog Patterns and LangSmith Insights derive cross-trace hierarchies and roll up metrics, NeMo profiles instrumented workflows, and OpenTelemetry Profiles links profiles to traces (Datadog 2026; LangChain 2026; NVIDIA 2026; OpenTelemetry 2026). These systems establish grouping, rollups, profile naming and compatibility, and trace linkage. AGENTPROF combines these capabilities in one operation corpus by joining agent actions to process, file, and network effects while conserving arbitrary additive measures and supporting selectable query-time field stacks.

Cross-run agent analysis. TraceProbe supplies resource-aware process profiles, while Graphectory builds per-trajectory process graphs with aggregated phase patterns (Shu et al. 2026; Liu et al. 2026a). Hodoscope compares behavior distributions to direct human review, while

TraceGraph builds shared decision landscapes that guide recovery (Zhong, Saxena, and Raghunathan 2026; Nian et al. 2026). Together, these systems establish canonical actions, recurring cross-run structure, and analysis-to-decision precedents. AGENTPROF complements them with source-linked heterogeneous effects, conservation of user-selected additive measures, and multiple operation-stack projections over the same histories.

Profiling and diagnosis. pprof and flame graphs fold run-time stacks, whereas Pivot Tracing dynamically selects and groups measurements across causally related events (Google 2024; Gregg 2011; Mace, Roelke, and Fonseca 2015). AgentRx diagnoses failures, CodeTracer reconstructs per-run states, and localization benchmarks score faulty steps, agents, or expert-supported perspectives (Barke et al. 2026; Li et al. 2026; Fan et al. 2026; Wang et al. 2026; Chen et al. 2026a; In et al. 2026). AGENTPROF applies the profiler’s population-level principle to agents by treating completed trajectories as samples and using query-time operation stacks to fold recurring semantic responsibility without code identity or runtime nesting.

Conclusion

Agent observability needs profiling, not only debugging. AGENTPROF represents heterogeneous agent actions and system effects as weighted operations and folds selectable query-time operation stacks across runs. Its profiles preserve linked effects, improve stage-partition agreement and localization MAP over raw action, and recover OSWorld-Human groups at 0.680 boundary F1 and 0.786 B³ F1. AGENTPROF builds a 27,765-operation profile in 1.17 seconds, making population profiling practical alongside per-run debugging.

References

- Abhyankar, R.; Qi, Q.; and Zhang, Y. 2025. OSWorld-Human: Benchmarking the Efficiency of Computer-Use Agents. arXiv preprint arXiv:2506.16042.
- Anthropic. 2025. Claude Code: An Agentic Coding Tool. <https://code.claude.com/docs/en/overview>.
- Arize AI. 2024. OpenInference Specification. <https://arize-ai.github.io/openinference/spec/>.
- Bagga, A.; and Baldwin, B. 1998. Entity-Based Cross-Document Coreferencing Using the Vector Space Model. In *36th Annual Meeting of the Association for Computational Linguistics and 17th International Conference on Computational Linguistics, Volume 1*, 79–85. Montreal, Quebec, Canada: Association for Computational Linguistics.
- Barke, S.; Goyal, A.; Khare, A.; Singh, A.; Nath, S.; and Bansal, C. 2026. AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. arXiv preprint arXiv:2602.02475.
- Bouma, G. 2009. Normalized (Pointwise) Mutual Information in Collocation Extraction. In *From Form to Meaning: Processing Texts Automatically, Proceedings of the Biennial GSCL Conference 2009*, 31–40. Tübingen, Germany.
- Bouzenia, I.; and Pradel, M. 2025. Understanding Software Engineering Agents: A Study of Thought-Action-Result

- Trajectories. In *2025 IEEE/ACM 40th International Conference on Automated Software Engineering (ASE)*, 2846–2857. IEEE.
- Chen, M.; Wang, J.; Mu, F.; Wang, Y.; Liu, Z.; Feng, H.; and Wang, Q. 2026a. Seeing the Whole Elephant: A Benchmark for Failure Attribution in LLM-Based Multi-Agent Systems. arXiv preprint arXiv:2604.22708.
- Chen, X.; Yin, Z.; He, S.; Huang, B.; Lei, S.; et al. 2026b. Safactory: A Scalable Agentic Infrastructure for Training Trustworthy Autonomous Intelligence. arXiv preprint arXiv:2605.06230.
- Datadog. 2025. LLM Observability. https://docs.datadoghq.com/llm_observability/.
- Datadog. 2026. LLM Observability Patterns. https://docs.datadoghq.com/llm_observability/monitoring/patterns/.
- Deng, X.; Gu, Y.; Zheng, B.; Chen, S.; Stevens, S.; Wang, B.; Sun, H.; and Su, Y. 2023. Mind2Web: Towards a Generalist Agent for the Web. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.
- Fan, S.; Ye, X.; Huo, Y.; Chen, Z.-Y.; Guo, Y.; Yang, S.; Yang, W.; Ye, S.; Chen, J.; Chen, H.; Cong, X.; and Lin, Y. 2026. AgentProcessBench: Diagnosing Step-Level Process Quality in Tool-Using Agents. In *Proceedings of KDD*.
- Gerganov, G.; and contributors. 2026. llama.cpp: LLM Inference in C/C++. GitHub repository.
- Google. 2024. pprof. <https://github.com/google/pprof>.
- Gregg, B. 2011. Flame Graphs. <https://www.brendangregg.com/flamegraphs.html>.
- In, Y.; Tanjim, M.; Subramanian, J.; Kim, S.; Bhattacharya, U.; Kim, W.; Park, S.; Sarkhel, S.; and Park, C. 2026. Rethinking Failure Attribution in Multi-Agent Systems: A Multi-Perspective Benchmark and Evaluation. arXiv:2603.25001.
- LangChain. 2024. LangSmith Observability. <https://docs.langchain.com/langsmith/observability>.
- LangChain. 2026. LangSmith Insights. <https://docs.langchain.com/langsmith/insights>.
- Langfuse. 2024. Langfuse Observability. <https://langfuse.com/docs/observability/overview>.
- Lewis, D. D.; Yang, Y.; Rose, T. G.; and Li, F. 2004. RCV1: A New Benchmark Collection for Text Categorization Research. *Journal of Machine Learning Research*, 5: 361–397.
- Li, H.; Yao, Y.; Zhu, L.; Feng, R.; Ye, H.; Wang, J.; He, Y.; Zou, P.; Zhang, L.; Lei, X.; Huang, H.; Deng, K.; Sun, M.; Zhang, Z.; Ye, H.; and Liu, J. 2026. CodeTracer: Towards Traceable Agent States. arXiv:2604.11641.
- Li, M.; Zhao, Y.; Yu, B.; Song, F.; Li, H.; Yu, H.; Li, Z.; Huang, F.; and Li, Y. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 3102–3116.
- Li, W.; Bishop, W. E.; Li, A.; Rawles, C.; Campbell-Ajala, F.; Tyamagundlu, D.; and Riva, O. 2024. On the Effects of Data Scale on UI Control Agents. In *NeurIPS 2024, Datasets and Benchmarks Track*.
- Liu, S.; Chen, Y.; Krishna, R.; Sinha, S.; Ganhotra, J.; and Jabbarvand, R. 2026a. Process-Centric Analysis of Agentic Software Systems. *Proceedings of the ACM on Programming Languages*, 10(OOPSLA1): 1961–1988.
- Liu, Z.; Xie, J.; Ding, Z.; Li, Z.; Yang, B.; et al. 2026b. ScaleCUA: Scaling Open-Source Computer Use Agents with Cross-Platform Data. In *Proceedings of ICLR*.
- Lu, Q.; Shao, W.; Liu, Z.; Meng, F.; Li, B.; Chen, B.; Huang, S.; Zhang, K.; Qiao, Y.; and Luo, P. 2025. GUIOdyssey: A Comprehensive Dataset for Cross-App GUI Navigation on Mobile Devices. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.
- Lù, X. H.; Kasner, Z.; and Reddy, S. 2024. WebLINX: Real-World Website Navigation with Multi-Turn Dialogue. In *Proceedings of ICML*, volume 235 of *Proceedings of Machine Learning Research*, 33007–33056. PMLR.
- Lù, X. H.; Kazemnejad, A.; Meade, N.; Patel, A.; Shin, D.; Zambrano, A.; Stańczak, K.; Shaw, P.; Pal, C. J.; and Reddy, S. 2025. AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories. In *Conference on Language Modeling (COLM)*.
- Ma, C.; Zhang, J.; Zhu, Z.; Yang, C.; Yang, Y.; Jin, Y.; Lan, Z.; Kong, L.; and He, J. 2024. AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents. In *Advances in Neural Information Processing Systems*, volume 37, 74325–74362. Curran Associates, Inc.
- Mace, J.; Roelke, R.; and Fonseca, R. 2015. Pivot Tracing: Dynamic Causal Monitoring for Distributed Systems. In *Proceedings of the 25th Symposium on Operating Systems Principles*, 378–393.
- MacQueen, J. B. 1967. Some Methods for Classification and Analysis of Multivariate Observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, 281–297. University of California Press.
- McCallum, A.; and Nigam, K. 1998. A Comparison of Event Models for Naive Bayes Text Classification. In *AAAI-98 Workshop on Learning for Text Categorization*, 41–48. AAAI Press.
- Nian, J.; Chen, K.; Zhang, G.; Cao, Y.; and Jiang, Y. 2026. TraceGraph: Shared Decision Landscapes for Diagnosing and Improving Agent Trajectories. arXiv:2605.31308.
- NVIDIA. 2026. Profiling and Performance Monitoring of NVIDIA NeMo Agent Toolkit Workflows. Official documentation.
- OpenAI. 2025. OpenAI Codex CLI. GitHub repository.
- OpenTelemetry. 2024. OpenTelemetry GenAI Semantic Conventions. <https://github.com/open-telemetry/semantic-conventions-genai>.
- OpenTelemetry. 2026. OpenTelemetry Profiles. Official specification.
- Qin, Y.; Liang, S.; Ye, Y.; Zhu, K.; Yan, L.; Lu, Y.; Lin, Y.; Cong, X.; Tang, X.; Qian, B.; Zhao, S.; Hong, L.; Tian, R.; Xie, R.; Zhou, J.; Gerber, M.; Li, D.; Liu, Z.; and Sun, M. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *Proceedings of the*

12th International Conference on Learning Representations (ICLR).

Qwen Team. 2026. Qwen3.6-27B: Flagship-Level Coding in a 27B Dense Model. Official model card.

Robertson, S. 2008. A New Interpretation of Average Precision. In *Proceedings of the 31st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 689–690. ACM.

Rosenberg, A.; and Hirschberg, J. 2007. V-Measure: A Conditional Entropy-Based External Cluster Evaluation Measure. In *Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning*, 410–420. Prague, Czech Republic: Association for Computational Linguistics.

Ruokolainen, T.; Kohonen, O.; Sirts, K.; Grönroos, S.-A.; Kurimo, M.; and Virpioja, S. 2016. A Comparative Study of Minimally Supervised Morphological Segmentation. *Computational Linguistics*, 42(1): 91–120.

Sajadi, A.; Nguyen, T.; Huynh, K.; Parra, E.; and Chatterjee, P. 2026. TraceView: Interactive Visualization of Agentic Program Repair Trajectories. arXiv:2606.22110.

Salton, G.; and Buckley, C. 1988. Term-Weighting Approaches in Automatic Text Retrieval. *Information Processing and Management*, 24(5): 513–523.

Shu, R.; Chong, C. Y.; Zhou, X.; Peng, Y.; Wu, Z.; Han, X.; Zhuang, Z.; Yuan, G.; and Wang, Y. 2026. What Resolve Rate Hides: Trajectory Structure Diagnostics for Coding Agents. arXiv:2607.06184.

Wang, J.; Hou, J.; Wang, F.; Jian, P.; Bao, C.; and Lv, Z. 2026. HINTBench: Horizon-Agent Intrinsic Non-Attack Trajectory Benchmark. arXiv preprint arXiv:2604.13954.

Wang, R.; Jansen, P.; Côté, M.-A.; and Ammanabrolu, P. 2022. ScienceWorld: Is Your Agent Smarter than a 5th Grader? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 11279–11298. Association for Computational Linguistics.

Wang, X.; Wang, B.; Lu, D.; Yang, J.; Xie, T.; et al. 2025. AgentNet. <https://huggingface.co/datasets/xlangai/AgentNet>.

Wilson, E. B. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association*, 22(158): 209–212.

Xie, T.; Zhang, D.; Chen, J.; Li, X.; Zhao, S.; Cao, R.; Hua, T. J.; Cheng, Z.; Shin, D.; Lei, F.; Liu, Y.; Xu, Y.; Zhou, S.; Savarese, S.; Xiong, C.; Zhong, V.; and Yu, T. 2024. OS-World: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track*.

Xu, Y.; Lu, D.; Shen, Z.; Wang, J.; Wang, Z.; Mao, Y.; Xiong, C.; and Yu, T. 2025. AgentTrek: Agent Trajectory Synthesis via Guiding Replay with Web Tutorials. In *Proceedings of ICLR*.

Yang, J.; Jimenez, C. E.; Wettig, A.; Lieret, K.; Yao, S.; Narasimhan, K.; and Press, O. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineer-

ing. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*.

Yao, S.; Chen, H.; Yang, J.; and Narasimhan, K. 2022. Web-Shop: Towards Scalable Real-World Web Interaction with Grounded Language Agents. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*.

Yao, S.; Shinn, N.; Razavi, P.; and Narasimhan, K. 2025. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *The Thirteenth International Conference on Learning Representations (ICLR)*.

Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E. P.; Zhang, H.; Gonzalez, J. E.; and Stoica, I. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.

Zheng, Y.; Hu, Y.; Yu, T.; and Quinn, A. 2025. AgentSight: System-Level Observability for AI Agents Using eBPF. In *Proceedings of the 4th Workshop on Practical Adoption Challenges of ML for Systems*.

Zhong, Z.; Saxena, S.; and Raghunathan, A. 2026. Hodoscope: Unsupervised Monitoring for AI Misbehaviors. arXiv:2604.11072.